Laboratorio S2 · 135 minutos

Se trabaja dentro de `04-Proyecto-Base/` con el entorno `env-ia-mlops` (terminal: **Anaconda Prompt** o la terminal de VS Code con el entorno activado). Los bloques 1 a 3 usan **dos terminales**: una para la API y otra para los comandos `curl` y las pruebas. El bloque 4 (Docker) se hace en un **Codespace**; el 5, en GitHub.

Los comandos valen en Linux, macOS y CMD. Donde PowerShell cambia, se dice. `make` no existe en Windows: se usan las líneas equivalentes.

Min	Bloque
0–10	0. Preparar
10–35	1. Arrancar la API
35–65	2. Romperla a propósito
65–75	**Pausa**
75–100	3. Comparar caminos y romper reglas
100–125	4. Docker en Codespaces
125–135	5. El primer workflow (CI)

Los números son los del **proyecto base con la muestra** (23 070 filas). La probabilidad del cliente de ejemplo debe ser `0.8278889191932133`; en otro sistema operativo los últimos decimales pueden variar.

0. Preparar (10 min)

Parte de `develop` limpio, no de la rama de la tarea de S1.

```
conda activate env-ia-mlops
cd 04-Proyecto-Base
python -m ia_mlops.pipeline
python -m pytest -q -m "not lento"
```

Esperado: `127 passed, 4 deselected`. Comprueba que existen `models/modelo_mlflow/` y `models/metadatos.json`: sin ellos la API no tiene qué servir.

1. Arrancar la API (25 min)

Terminal 1 (déjala abierta; aquí verás el registro):

```
python -m uvicorn ia_mlops.api.app:app --port 8080
```

(`make api` hace lo mismo.) Debe terminar con `Uvicorn running on http://127.0.0.1:8080`.

Terminal 2 (también con el entorno activado y dentro de `04-Proyecto-Base`):

```
curl http://127.0.0.1:8080/salud
curl http://127.0.0.1:8080/listo
curl http://127.0.0.1:8080/modelo
curl -X POST http://127.0.0.1:8080/predecir -H "content-type: application/json" -d
"@tests/datos/cliente_ejemplo.json"
```

Esperado: `{"estado":"ok"}`, `{"estado":"listo"}`, la ficha del modelo y `{"probabilidad":0.8278889191932133,"modelo":"regresion_logistica","huella_datos":"35a94deb4e04ab4a","advertencias":[]}`.

PowerShell: escribe `curl.exe` en vez de `curl` y deja las comillas dobles alrededor de `@ruta` (sin ellas PowerShell interpreta la `@`).

Haz:

1. Abre http://127.0.0.1:8080/docs. Despliega `POST /predecir`, pulsa *Try it out* y *Execute* con el ejemplo que trae. ¿Coincide con `curl`?
2. Un lote de dos clientes (el ejemplo y el caso de una sola factura):

   ```
   curl -X POST http://127.0.0.1:8080/predecir/lote -H "content-type: application/json" -d
   "@tests/datos/lote_ejemplo.json"
   ```

   Esperado: dos predicciones, en el mismo orden (`0.8278889191932133` y `0.30612919598959804`); la segunda lleva una advertencia (`dias_medios_entre_compras no informado`).
3. Mira la Terminal 1: cada petición dejó **una línea JSON**.

Preguntas: ¿qué diferencia hay entre `/salud` y `/listo`? ¿Por qué `/modelo` devuelve `huella_datos`?

2. Romperla a propósito (30 min)

Primero, los **diez casos** de `tests/datos/casos/`. Antes de cada uno, **apunta qué código esperas** (200, 422 o 503) y por qué; después lanza:

```
curl -i -X POST http://127.0.0.1:8080/predecir -H "content-type: application/json" -d
"@tests/datos/casos/02_recencia_negativa.json"
```

(cambia el nombre del archivo en cada caso). `-i` muestra las cabeceras: busca `x-request-id`.

Archivo	Qué tiene de particular	Código	Mensaje clave
`01_correcto.json`	el ejemplo	200	`probabilidad`
`02_recencia_negativa.json`	`recencia_dias: -1`	422	`Input should be greater than or equal to 0`
`03_con_el_objetivo_y.json`	trae el objetivo `y`	422	`Extra inputs are not permitted`
`04_falta_frecuencia.json`	no trae `frecuencia`	422	`Field required`
`05_reino_unido_2.json`	`es_reino_unido: 2`	422	`Input should be 0 or 1`

Archivo	Qué tiene de particular	Código	Mensaje clave
`06_frecuencia_90d_mayor_que_total.json`	9 facturas en 90 días y 6 en total	422	`debe cumplirse frecuencia_90d <= frecuencia_180d <= frecuencia`
`07_seis_facturas_sin_dias_medios.json`	6 facturas y `dias_medios_entre_compras: null`	422	`es null si y solo si hay una sola factura`
`08_ticket_medio_enorme.json`	`ticket_medio: 5000` (el modelo vio hasta 1 294,87)	**200 con advertencia**	`fuera del rango visto al entrenar`
`09_una_sola_factura.json`	una factura, sin `dias_medios_entre_compras`	**200 con advertencia**	`no informado: el modelo usa la mediana`
`10_frecuencia_cero.json`	`frecuencia: 0` (cliente sin historia)	422	`Input should be greater than or equal to 1`

Preguntas: ¿en qué casos se equivocó quien llama? ¿Por qué los casos 08 y 09 **no** se rechazan? ¿Por qué `y` se rechaza en vez de ignorarse?

Después, el **servicio** también falla. Detén la API (Ctrl+C) y haz cada prueba por separado:

a) Sin modelo (503). Apunta a una carpeta vacía; no borres tu `models/`.

```
mkdir vacio
```

```
Linux / macOS :  CURSO_MODELOS=vacio python -m uvicorn ia_mlops.api.app:app --port 8080
CMD           :  set CURSO_MODELOS=vacio   y luego   python -m uvicorn ia_mlops.api.app:app --port
8080
PowerShell    :  $env:CURSO_MODELOS="vacio"   y luego   python -m uvicorn ia_mlops.api.app:app --
port 8080
```

Comprueba `/salud`, `/listo` y `/predecir`. Esperado: `/salud` **200** (el proceso está vivo) y `/listo` y `/predecir` **503** con `No hay modelo en … Entrenalo antes: python -m ia_mlops.pipeline`. Fíjate en que el 503 trae un `id_peticion` y que la Terminal 1 registra `modelo no cargado` con el motivo. Cierra la API y borra la variable (CMD: `set CURSO_MODELOS=`; PowerShell: `Remove-Item Env:CURSO_MODELOS`).

b) El contrato del modelo no coincide con sus metadatos (503). Copia la carpeta `models` como `models_roto` (con el explorador de archivos, o `cp -r models models_roto` en Linux/macOS, o `xcopy models models_roto /E /I` en CMD). Abre `models_roto/metadatos.json` y, en la lista `variables`, **intercambia** los nombres `recencia_dias` y `frecuencia`. Arranca con `CURSO_MODELOS=models_roto` (como en *a*) y mira `/listo`. Esperado: **503**, «La firma del modelo [...] no coincide con metadatos.json [...]». Es mejor fallar al desplegar que devolver probabilidades con las columnas cambiadas.

c) El esquema de la API no coincide con la firma (503). Es la situación de quien reentrena con una variable nueva y olvida la API. En `ia_mlops/api/esquemas.py` borra la línea del campo `devoluciones` en la clase `Cliente`, arranca la API (sin `CURSO_MODELOS`) y mira:

- `/listo` → **503** con «Faltan en la API: ['devoluciones']» y qué archivo tocar.
- `POST /predecir` con el ejemplo → **422** `Extra inputs are not permitted` en `devoluciones`: la API ya no conoce ese campo y lo rechaza antes de mirar si el modelo está listo.

Restáuralo: `git restore ia_mlops/api/esquemas.py`.

d) Sigue una petición por el registro.

```
curl -i -X POST http://127.0.0.1:8080/predecir -H "content-type: application/json" -H "X-Request-ID: mi-prueba-1" -d "@tests/datos/casos/02_recencia_negativa.json"
```

Esperado: la cabecera `x-request-id: mi-prueba-1` y, en la Terminal 1, una línea con `"id": "mi-prueba-1"` y `"codigo": 422`. **Pregunta:** ¿hay algún dato del cliente en esa línea? (No debe haberlo.)

Pausa (10 min)

3. Comparar caminos y romper reglas (25 min)

Con la API en marcha (Terminal 1, sin variables raras):

```
python -m ia_mlops.modeling.comparar
python -m ia_mlops.modeling.comparar --url http://127.0.0.1:8080
```

Esperado (cien filas reales de la partición test):

```
camino                                  filas     diferencia maxima  resultado
paquete (referencia)                      100             0.000e+00  coincide
joblib                                    100             0.000e+00  coincide
MLflow                                    100             0.000e+00  coincide
API http://127.0.0.1:8080                 100             0.000e+00  coincide
Todos los caminos dan la misma probabilidad.
```

Una línea `INFO mlflow.agent.hint …` al principio es de MLflow y puede ignorarse. El código de salida es 0 si todo coincide.

Preguntas: ¿qué tendría que pasar para que una fila diera diferencia distinta de cero? (Otra versión de una librería, otro orden de columnas, otro preproceso.) ¿Por qué comparar con filas **reales** y no con una inventada?

Ahora **rompe una regla del esquema y mira qué prueba lo detecta**. Una por vez, en `ia_mlops/api/esquemas.py`; después de cada una ejecuta `python -m pytest tests/test_api.py -q -x` y restaura con `git restore ia_mlops/api/esquemas.py`:

Cambio	Prueba que debe fallar
En `frecuencia`, cambia `ge=1` por `ge=0`	`test_un_cliente_sin_historia_no_tiene_variables`
En `recencia_dias`, cambia `ge=0` por `ge=-10000`	`test_rechaza_valores_fuera_de_dominio`
En `model_config`, quita `extra='forbid',`	`test_rechaza_campos_que_no_son_del_contrato`

Anota en tu incidencia cuál falló y qué mensaje daba. **Pregunta:** ¿qué pasaría si nadie hubiera escrito esa prueba?

4. Docker en Codespaces (25 min)

Docker no se instala en tu equipo. Abre un **Codespace** del repositorio del curso (rama `develop`; botón verde *Code* → *Codespaces*). Antes de la clase, repasa la guía en diapositivas `03-Entorno/guia-codespaces/index.html` (en texto: `codespaces-para-el-alumnado.md`): cómo abrirlo en el navegador o desde tu VS Code, qué hacer si falla y cuánta cuota gastas. Si Docker es nuevo para ti, haz antes el autoestudio de contenedores (`guia-practica.md` y sus dos presentaciones; unos 60 min como mínimo). En su terminal:

```
cd 04-Proyecto-Base
docker version
```

Deben aparecer **Client** y **Server**. Si `docker` no existe, sigue con el bloque 5: el CI construye la imagen por ti.

```
python -m ia_mlops.pipeline
docker build -f deploy/docker/Dockerfile -t retencion-api:local .
docker image ls retencion-api:local
docker run -d --name api -p 8080:8080 retencion-api:local
curl http://127.0.0.1:8080/listo
```

Si `curl` da `Connection reset by peer` justo después de `docker run`, espera unos segundos y repite: el puerto ya está publicado, pero la API aún no ha arrancado.

```
python -m ia_mlops.modeling.comparar --url http://127.0.0.1:8080
docker logs api
docker exec api id -u
docker rm -f api
```

Esperado: la comparación coincide también contra el **contenedor**; `docker logs` muestra las mismas líneas JSON; `id -u` imprime `10001` (no es root).

Preguntas: ¿cuánto ocupa la imagen? ¿Qué pasa con la construcción si cambias una línea de `ia_mlops/` y vuelves a ejecutar `docker build`: qué capas se reutilizan? ¿Por qué el contenedor escucha en `0.0.0.0` y no en `127.0.0.1`?

Estado de esta práctica: el Dockerfile y los comandos se verificaron en GitHub Actions y en un Codespace real del curso (12/11/2026, máquina de 2 núcleos; capturas en la guía de Codespaces). Si algo difiere, **anótalo con el mensaje exacto**: es información valiosa.

5. El primer workflow (10 min)

La plantilla es `deploy/github-actions/ci.yml`. Para probarla necesitas un repositorio **tuyo** en GitHub:

1. Copia `deploy/github-actions/ci.yml` a `.github/workflows/ci.yml` en tu repositorio. Si tu repositorio es el clon del curso (el proyecto está en `04-Proyecto-Base/`), cambia en el archivo `PROYECTO: "."` por `PROYECTO: "04-Proyecto-Base"`.
2. Sube una rama `develop` (el workflow se ejecuta en `push` a `develop` o `main`).
3. En GitHub, pestaña **Actions**: abre la ejecución y despliega **Probar el contenedor**. Debe mostrar `/listo`, `/modelo`, la predicción y terminar sin error.

El workflow no publica nada ni usa secretos. **Pregunta:** ¿qué pasos del workflow ya habías hecho a mano hoy?

Si algo falla

Síntoma	Qué hacer
`COPY failed … models/modelo_mlflow` al construir	Falta entrenar: `python -m ia_mlops.pipeline` antes de `docker build`.
`/listo` da 503	Lee el `detalle`: falta el modelo, o el esquema/los metadatos no coinciden con la firma.
`Address already in use` / el puerto 8080 no responde	Otra API (o contenedor) ya lo usa: ciérrala o usa `--port 8081`.
El navegador no llega al contenedor	Falta `-p 8080:8080`, o la API escucha en `127.0.0.1` dentro del contenedor.
422 por un campo que «sobra»	Es el contrato: el objetivo `y`, `cliente_id` o la fecha no viajan. Quítalo.
`curl` en PowerShell falla con `@`	Usa `curl.exe` y comillas dobles: `-d "@ruta"`.
Una prueba falla y no sabes por qué	Ejecuta solo esa: `python -m pytest tests/test_api.py -q -x -k nombre`. Anota el mensaje en tus incidencias; no lo ocultes.

Para quien termina antes

Empieza la tarea: `../tarea/README.md`. Lo más útil: reentrenar con una variable nueva y ver cómo la API lo detecta en `/listo`.

Laboratorio extra (si sobra tiempo o el grupo lo pide): de la ingesta a la explotación. Recorre los pipelines uno a uno, pone el modelo en producción, saca la lista de la campaña puntuando a todos los clientes y cierra el ciclo con datos nuevos.